Adaptive and Explainable Fraud Detection Under Concept Drift

A Regression-Based Study on the IEEE-CIS Fraud Detection Dataset

Kanishk Bhuradiya (BSD-DH-2414) | Adarsh Kumar (BSD-DH-2403)

Regression Techniques | Prof. Minerva Mukhopadhyay | 6 November 2026

Abstract

Fraud detectors are usually evaluated with a random train–test split, which silently assumes that the relation between transaction features and fraud never changes. We drop this assumption. On the 590 540 labelled transactions of the IEEE-CIS data (26 weeks, 3.5% fraud) we build a leakage-controlled, chronological evaluation and ask how much accuracy is lost on future weeks, which features drift, which models are robust, how often to retrain, what retraining is worth in money, and whether explanations stay stable.

A random split overstates PR-AUC by 56% (0.816 vs. 0.523). Frozen tree models lose 15–20% of their PR-AUC between the first and last future weeks, and retraining on recent labels recovers about two thirds of the loss, lowering estimated cost by 7–17% when done every 2 or 4 weeks. The dominant input drift is a change in feature *availability*, not in values, and label-free monitors cannot see the accuracy loss, which we explain formally and show on the data. Because this is a regression course, we study the logistic-regression family in depth: penalties, binning and a tree–regression hybrid are compared through a bias–variance argument, and the drift itself is modelled with Poisson/negative-binomial regression and window-wise coefficient tests. Missing interactions, not regularisation, limit linear models here.

Contents

1 Introduction
2 Related work
3 Data and preprocessing
4 Theory and methods
5 Experimental setup
6.1 Evaluation protocol and baselines
6.2 What drifted?
6.3 Retraining and label delay
6.4 Financial cost
6.5 Explanation stability
6.6 The regression family and why trees win
6.7 Regression models of the drift
7 Discussion and observation table
8 Limitations 9 Conclusion

1 Introduction

Card-not-present fraud changes continuously: attackers adapt, merchants change what data they collect, and the customer mix shifts. A classifier trained on last quarter may therefore be wrong today even if it was excellent in the lab. The standard remedy is to retrain, but retraining costs money and, in fraud, labels arrive late because chargebacks take weeks [4]. This project measures the problem and the remedy on a public benchmark.

Research questions

RQ1 How much does detection performance deteriorate under temporal drift?

RQ2 Which features drift the most?

RQ3 Which models are the most robust?

RQ4 How often should the model be retrained?

RQ5 Does adaptive retraining reduce the estimated financial cost?

RQ6 Are explanations stable before and after drift?

RQ7 *(added for the regression course)* Why do some regression-type models beat others on skewed data, and can a penalised or hybrid logistic regression close the gap?

Contributions

- A leakage-controlled walk-forward protocol with label delay and a cost model (Sections 3–5).
- A formal argument, with matching evidence, for why score- and feature-based monitors can be blind to concept drift (Proposition 1, Section 6.2).
- A theory-guided comparison of the logistic-regression family with tree ensembles, including a tree+regression hybrid in the spirit of He et al. [10] (Section 6.6).
- Regression models of the drift itself: Poisson/negative-binomial count models with change points and window-wise coefficient tests (Section 6.7).

We do not propose a new algorithm; the value lies in the protocol, the measurements and the explanations, each labelled with its evidence level (Section 7).

2 Related work

Concept drift is a change over time in the relation between inputs and target [9]. The survey catalogues adaptation strategies (windowing, retraining, detectors such as DDM [8]) and the chronological test-then-train protocol we follow. Dal Pozzolo et al. [4, 5] study real card-fraud streams and stress that labels arrive days late and that alert precision matters to investigators; our label-delay experiment follows this setting in simplified form.

Regression tools. The lasso [17] and elastic net [18] penalise coefficient size. He et al. [10] feed the leaves of boosted trees into a sparse linear classifier. Weight-of-evidence binning is the credit-scoring standard [16], and count outcomes are modelled by Poisson and negative-binomial regression [2]. Explanations use Shapley values [14]. Forests and boosted trees follow Breiman [1], Friedman [7], Chen and Guestrin [3] and Ke et al. [12].

3 Data and preprocessing

Source. The IEEE-CIS Fraud Detection data (Vesta Corporation) are public on Kaggle [11]. We use the two labelled files `train_transaction.csv` (590 540 rows, 394 columns) and `train_identity.csv` (144 233 rows, 41 columns), left-joined on `TransactionID` (434 columns). The test files have no public labels and are not used. The data are not bundled with this report because the competition terms forbid redistribution.

Content. Each row is a transaction with amount, product code, card and address fields, e-mail domains, count features $C1$–$C14$, time-delta features $D1$–$D15$, match flags $M1$–$M9$, 339 masked

engineered features $V1$–$V339$ and, for 24% of rows, identity/device fields. `TransactionDT` is in seconds from an unknown origin; we use day $=$ `TransactionDT`$/86400 - 1$ and week $= \lfloor \text{day}/7 \rfloor$, giving weeks 0–25. There are 20 663 frauds (3.50%).

3.1 Time design

Rows are sorted by time. Week 0 is a warm-up, weeks 1–12 are training, weeks 13–14 are validation (early stopping and thresholds) and weeks 15–25 form the "future" test, scored one week at a time (Figure 1, Table 1).

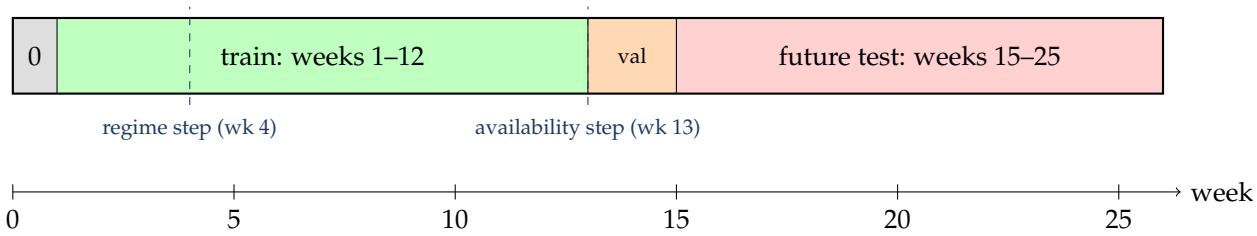


Figure 1. Chronological protocol. Every model only sees rows earlier than the week it scores (minus the label delay in the delay experiment).

Table 1. Time-based splits.

Period	Days	Rows	Frauds	Fraud rate
Training	7–91	292 338	10 078	3.45%
Validation	91–105	48 698	1 602	3.29%
Future test	≥ 105	221 908	8 179	3.69%

3.2 Preprocessing decisions

Missing values are kept. Missingness is informative: when `addr1` is missing the fraud rate is 11.8% versus 2.5% when present; for the 93%-empty column $D7$ it is 14.9% when present versus 2.7% when missing; rows with identity data have a 7.8% fraud rate versus 2.1% without. Dropping incomplete rows would discard most of the data and the signal. Tree models use missing values natively; the linear models receive median imputation plus explicit missing-indicator columns (Section 6.6).

Column selection (fitted on the training window only).

- Drop 9 identity columns that are $> 95\%$ empty and 2 constant columns ($V1$, $V107$).
- Drop 213 of 337 V columns whose absolute correlation with an earlier kept V column exceeds 0.95 (124 kept), leaving **207 features**.
- Code the 29 text columns by order of first appearance in time, so a code never depends on the future; `TransactionID` and `TransactionDT` are never inputs.

Class imbalance is handled by class weights inside the models. SMOTE is avoided because interpolating between NaN-heavy, partly categorical rows is not meaningful.

4 Theory and methods

This section states only the formal tools and the claims they support; each claim is tested in Section 6.

4.1 Drift types and why monitors can be blind

Write the joint law at time t as $P_t(x, y) = P_t(y \mid x)P_t(x)$. *Prior drift* changes $P_t(y)$, *covariate drift* changes $P_t(x)$ and *concept drift* changes $P_t(y \mid x)$ [9]. A deployed model computes a score $s = f(x)$ with f frozen.

Proposition 1 (Score monitoring is blind to pure concept drift)**.** *If $P_t(x) = P_0(x)$, the score law $P_t(s \in A) = P_0(f(x) \in A)$ is unchanged for every $P_t(y \mid x)$. Any statistic of the score distribution (mean, PSI, KS) is therefore constant in t, while precision, recall and AP can change arbitrarily.*

The score is a push-forward of $P_t(x)$ alone; y never enters. The same holds for any fixed function of x, such as the top features. Only labels reveal concept drift, which motivates label-based retraining triggers (Section 4.5).

4.2 Metrics under imbalance

With prevalence π, true-/false-positive rates TPR and FPR, precision is

$$\mathrm{Prec} = \frac{\pi\,\mathrm{TPR}}{\pi\,\mathrm{TPR} + (1-\pi)\,\mathrm{FPR}}. \tag{1}$$

At $\pi = 0.035$, an FPR of 1% with TPR of 60% gives precision ≈ 0.69, so ROC-AUC hides the false-alarm burden [15]. We therefore use average precision (called PR-AUC), whose random baseline is $\pi \approx 0.035$, together with lift $= \mathrm{AP}/\pi$, ROC-AUC and Recall@K (share of frauds in the top K% of scores per week, a review budget). Bootstrap intervals use resampled transactions.

4.3 Drift measures

For a feature with reference bin masses p_i (training-window deciles plus one "missing" bin) and current masses q_i,

$$\mathrm{PSI} = \sum_i (q_i - p_i) \ln \frac{q_i}{p_i} = \mathrm{KL}(q\|p) + \mathrm{KL}(p\|q), \tag{2}$$

a symmetrised Kullback–Leibler divergence [13]. Credit practice reads PSI > 0.1 as moderate and > 0.25 as large. Because PSI is hypersensitive for narrow or autocorrelated features, we estimate a per-feature *noise floor* ν_j (95th percentile of each training week against the rest of the training window) and report the excess $\max(\mathrm{PSI} - \nu_j, 0)$. We complement PSI with the Kolmogorov–Smirnov statistic and the Wasserstein-1 distance (scaled by the reference IQR), both computed on observed values only. PSI contains the missing-bin mass while the other two ignore it, so the three measures answer different questions.

4.4 Why linear models plateau

Logistic regression models $P(y{=}1 \mid x) = \sigma(\beta_0 + \beta^\top x)$ and minimises the penalised log-loss

$$-\frac{1}{n}\sum_i \left[y_i \ln p_i + (1-y_i)\ln(1-p_i)\right] + \lambda\left[\tfrac{1-\alpha}{2}\|\beta\|_2^2 + \alpha\|\beta\|_1\right], \tag{3}$$

which is ridge for $\alpha = 0$, lasso for $\alpha = 1$ and elastic net in between [17, 18]. The excess risk of any fitted model splits into an *approximation* term (how far the best member of the model class is from the truth) and an *estimation* term (how far the fitted model is from the best member). Penalties act only on the estimation term; they cannot change the model class.

The logit is additive, which gives a concrete limit:

Proposition 2 (Additive logits cannot express interactions). *Let x_1, x_2 be i.i.d. Bernoulli($\frac{1}{2}$) and $y = x_1 \oplus x_2$. The unique maximiser of the additive logistic likelihood is $\beta = 0$ (constant scores, AUC $= \frac{1}{2}$), while a depth-2 tree attains AUC $= 1$.*

A depth-d tree tests at most d features per path, so a boosted sum of depth-d trees has interaction order $\leq d$ [7], and depth-1 boosting is an additive model. **Prediction:** depth-1 boosting $\approx$ logistic regression, and gains appear at depth 2–3.

Tree–regression hybrid. Let $\phi(x)$ be the one-hot vector of the leaves reached in each boosted tree. The hybrid fits a penalised logistic regression on $\phi(x)$ [10]. Each column is a conjunctive rule, so the linear model has interactions up to the tree depth. Boosting already fits the leaf values by gradient steps, so refitting them jointly should help weak tree sets but may add nothing for strong ones.

Weight of evidence and Naive Bayes. WoE binning scores each bin by $\ln(b_i/g_i)$ (fraud share over normal share), and its information value is the same symmetrised-KL form as PSI. Naive Bayes adds log-likelihood ratios per feature, so duplicated features are counted twice; with 124 strongly correlated V columns it should be over-confident and badly ranked.

4.5 Class weights, thresholds, cost and retraining

A positive-class weight w multiplies the fitted odds by w, so rankings (AP) are unchanged but scores are not calibrated; thresholds are therefore chosen empirically on validation data. For threshold τ and transaction amounts a_i the cost is

$$C(\tau) = \sum_i \mathbb{1}[s_i \geq \tau](1 - y_i)(c_{fp} + \rho a_i) + \mathbb{1}[s_i < \tau]\, y_i(\kappa a_i + c_{fn}), \tag{4}$$

and with calibrated p flagging is optimal iff $\frac{p}{1-p} > \frac{c_{fp}+\rho a}{\kappa a + c_{fn}}$ [6]. The *break-even cost* of one extra retrain is $(C_{\text{static}} - C_s)/(R_s - 1)$, where R_s is the number of trainings of strategy s.

Walk-forward retraining. At the start of test week w a strategy may retrain using only labels available at that time (days $< t_w - d$ for label delay d; $d = 0$ unless stated). Strategies: *static* (train once), *fixed* (every k weeks on all data), *rolling* (last 8 weeks, every 2 weeks) and *triggered* (retrain if $1 - \text{AP}_{w-1}/\text{AP}_{\text{val}} > \theta$, with a two-week cooldown; $\theta \in \{0.10, 0.15, 0.25\}$, all reported and none selected on the test weeks).

4.6 Explanation stability

SHAP values [14] are computed exactly for trees. Importance is the mean $|\varphi_j|$ per feature; stability between two periods is the Spearman correlation of the importance vectors and the Jaccard overlap of the top-20 sets, always compared with a noise baseline (a new sample of the same period, or a model with a different seed).

4.7 Regression models of the drift itself

Count models. Weekly frauds $Y_t \sim \text{Poisson}(n_t \lambda_t)$ with $\ln E[Y_t] = \ln n_t + \beta_0 + \beta_1 t + \gamma_1 \mathbb{1}[t \geq 4] + \gamma_2 \mathbb{1}[t \geq 13]$, where $\ln n_t$ is an offset and e^{β_1} is a weekly rate ratio. Counts are heavily overdispersed, so inference uses quasi-Poisson or negative-binomial versions [2]. Since the fraud rate is $F/(F+L) \approx F/L$ for legitimate count L, $\ln(\text{rate ratio}) \approx \ln(\text{fraud ratio}) - \ln(\text{legitimate ratio})$, which separates a rising numerator from a falling denominator.

Coefficient drift. The same logistic regression (fixed design) is refitted on consecutive 4-week windows. The sampling noise $\hat{\sigma}$ of a coefficient is estimated from two random halves of one window, and a coefficient is flagged as changed between windows when $|\hat{\beta}^{(k)} - \hat{\beta}^{(1)}|/(\sqrt{2}\,\hat{\sigma}) > 3$. Because the halves ignore repeated cards, $\hat{\sigma}$ is optimistic.

5 Experimental setup

- **Models.** LightGBM [12], XGBoost [3], Random Forest [1] and logistic regression (median imputation, quantile-normal scaling, one-hot for text columns with ≤ 30 values), all with class weights. Key settings: LightGBM 64 leaves, colsample 0.5, subsample 0.8, positive weight 5, early stopping on AP; XGBoost depth 8, lr 0.05; Random Forest 200 trees, min leaf 5; logistic regression $C = 0.1$ (variants in Section 6.6). Seeds are fixed (42; 7 for the explanation baseline; 1–5 for the seed test). Full settings are in the code repository.
- **Two LightGBM versions.** The *first* LightGBM (lr 0.05, up to 1000 trees) is the frozen baseline. The retraining, delay, seed and SHAP experiments use a faster version (lr 0.1, up to 500 trees), called *static* when it is trained once.
- **Thresholds** (F1-optimal or cost-optimal) are chosen on each model's own validation weeks.
- **Seeds.** Five LightGBM seeds on identical data give the run-to-run noise.
- **Costs** use four assumed scenarios (Table 2); the data contain no cost information.
- **Software.** Python (pandas, scikit-learn, LightGBM, XGBoost, SciPy, statsmodels) on Kaggle notebooks; code is on GitHub (link after the references).

Table 2. Assumed cost scenarios (amounts in the dataset's currency units, treated as dollars).

Scenario	False alarm	Missed fraud
cheap review	\$2	100% of amount
balanced	\$10	100% of amount + \$15
friction-heavy	\$25 + 2% of amount	100% of amount + \$15
fraud-heavy	\$5	200% of amount + \$25

6 Results

6.1 Evaluation protocol and baselines (RQ1, RQ3)

Random versus chronological evaluation. The same LightGBM configuration scores PR-AUC 0.816 on a random 70/10/20 split but only 0.523 on the future weeks, so the random protocol overstates performance by 56% (Figure 2a). Its validation score was 0.549. A random model scores $\pi \approx 0.035$, so the future-test lift is about 15.

Decay of frozen models. Table 3 shows that every model degrades between the first (weeks 15–17) and last (weeks 23–25) future weeks. For LightGBM the drop is 0.124 (bootstrap mean

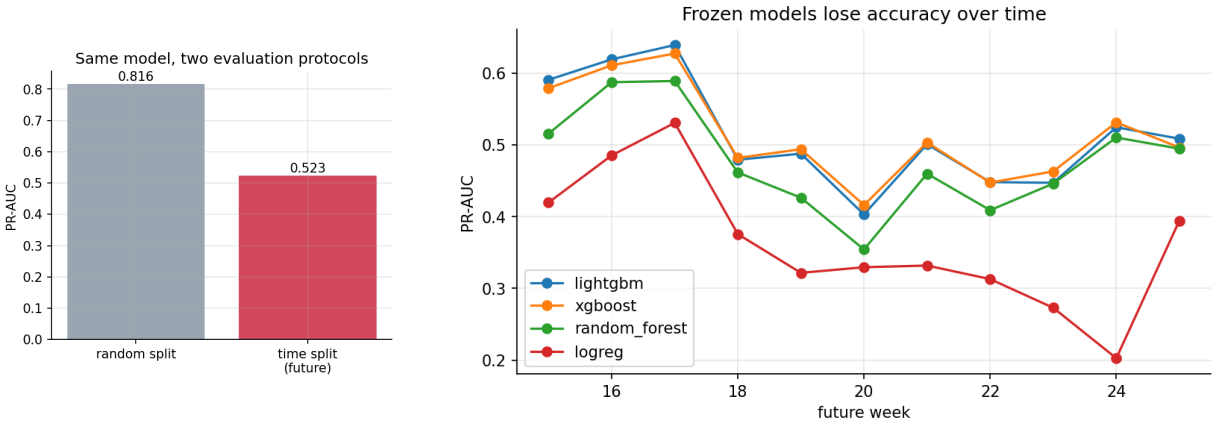


Figure 2. (a) Same model, random split versus time split. (b) Weekly PR-AUC of four frozen models over the future weeks; logistic regression (without missing indicators) is lowest.

0.122, 95% interval 0.096–0.151). To check that this is not a lucky seed, we retrained the static model with five seeds: PR-AUC over all weeks has standard deviation 0.004 (range 0.505–0.515), and the early-minus-late difference is positive for all five (0.116 ± 0.007; range 0.108–0.128). The drop is therefore more than ten times the seed variation, while differences of ≤ 0.004 (LightGBM vs. XGBoost) are not meaningful.

Table 3. Frozen baselines on the future weeks (trained once on weeks 1–12). Drop = weeks 15–17 minus weeks 23–25.

Model	ROC-AUC	PR-AUC	wk 15–17	wk 23–25	drop	rel. drop
LightGBM	0.894	0.523	0.617	0.493	0.124	20%
XGBoost	0.894	0.524	0.607	0.499	0.107	18%
Random Forest	0.885	0.487	0.564	0.483	0.082	15%
Logistic regression (no missing flags)	0.860	0.348	0.482	0.261	0.221	46%

Observations

- A random split makes the model look 56% better than it is on the future (0.816 vs. 0.523).
- All four models decay; the boosters lose about 18–20%, Random Forest 15%, plain logistic regression 46%.
- The 0.9 scores often quoted for this dataset correspond to ROC-AUC (0.894) or to random splits, not to future PR-AUC.

6.2 What drifted? (RQ2)

Input drift is mostly availability. Using PSI above the noise floor, 17 of 207 features are large drifters (PSI > 0.25). They form only three or four independent groups, because the columns of a group are filled or left empty together: $M1$–$M3$, $M7$–$M9$, $V2$–$V10$ and $D11$ (Table 4). The share of missing values in these groups falls from about 70% to about 30%, with a second step at week 13 (Figure 3); the control column `addr1` stays flat.

KS and Wasserstein, which ignore missing values, show almost no change in the values of the flagged group (median excess 0.000 and 0.004) while the missing rate moves by -30.4 points; the largest KS excess in the whole table is only 0.057. The three measures rank features differently (Spearman 0.16 PSI–KS, 0.18 PSI–Wasserstein, 0.59 KS–Wasserstein), consistent with the different quantities they measure. Figure 4 shows that the flagged features drift in almost every future week, with an exception in weeks 18–19 for $D11$ and the V group.

Table 4. Drifting feature groups: mean PSI over future weeks, noise floor ν, excess, KS/Wasserstein excess (observed values only) and change of missing rate (test minus training, percentage points).

Group	PSI	ν	excess	KS exc.	W1 exc.	Δ missing
*M*7–*M*9	0.516	0.163	0.354	n/a (text)	n/a	–
*M*1–*M*3	0.451	0.111	0.339	n/a (text)	n/a	–
*D*11	0.428	0.114	0.314	0.043	0.086	−30.4
*V*2–*V*10 (e.g. *V*4)	0.399	0.113	0.286	0.000	0.000	−30.4

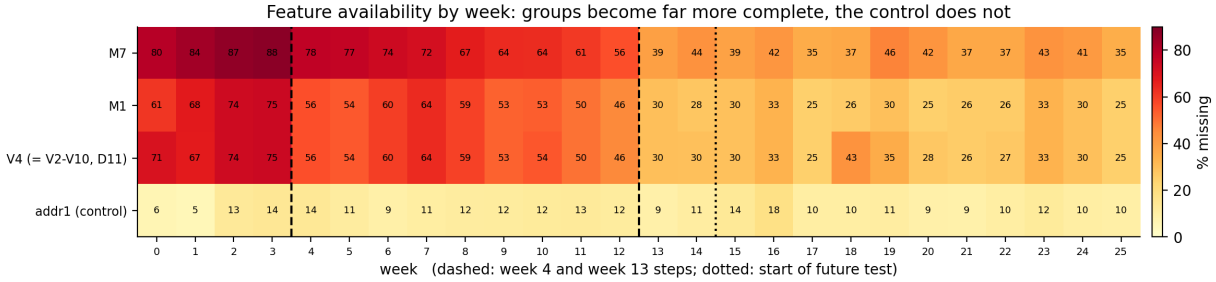


Figure 3. Share of missing values per week. Dashed lines: weeks 4 and 13; dotted: start of the future test. Several groups become far more complete, the control (`addr1`) does not.

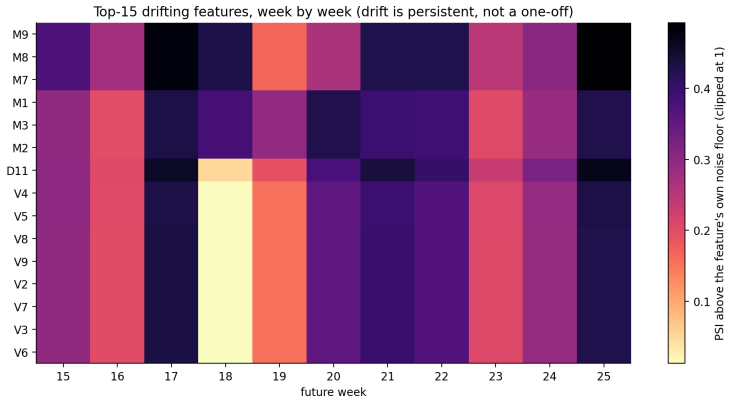


Figure 4. Top-15 drifting features, week by week (PSI above each feature's own noise floor, clipped at 1).

The link to fraud changed inside the drifting groups. Table 5 shows that the overall fraud rate hides a change in composition: the "$V4$ missing" group shrank from 66% to about 28% of transactions and its fraud rate more than doubled (3.6% → 7.3–7.9%), whereas the "present" group stayed at about 2%. The first LightGBM keeps its lift (9.1) in the missing group, but in the present group the lift falls from 10.8 to 6.0 (PR-AUC 0.221 → 0.115). This split was chosen after seeing the drift results, so it is a lead, not a confirmed mechanism.

Table 5. $V4$ missing vs. present. Fraud rates in %. Right: PR-AUC (lift) of the first LightGBM on the two groups.

Period	missing	fraud | miss.	fraud | pres.	Future group	PR-AUC	lift
train wk 1–6	66.0%	3.56	1.69	present, wk 15–19	0.221	10.8
train wk 7–12	54.0%	5.79	2.18	present, wk 20–25	0.115	6.0
validation	29.9%	6.23	2.04	missing, wk 15–19	0.720	9.1
test wk 15–19	32.9%	7.89	2.05	missing, wk 20–25	0.664	9.1
test wk 20–25	28.3%	7.31	1.91			

Label-free monitors were silent, as Proposition 1 predicts. The PSI of the static model's scores stayed between 0.005 and 0.020 in every future week and is unrelated to weekly PR-AUC (Spearman 0.05, $p = 0.87$). The PSI of its 20 most important features, above their noise floors, was 0.000–0.005 (Spearman 0.15, $p = 0.66$), although PR-AUC fell from about 0.6 to about 0.43–0.47. Figure 5 shows why: the score distribution of all transactions barely moves (KS 0.023), while the scores of the *frauds* shift towards lower values (KS 0.104). The model has become less sure about frauds, and a monitor that does not see labels cannot notice this.

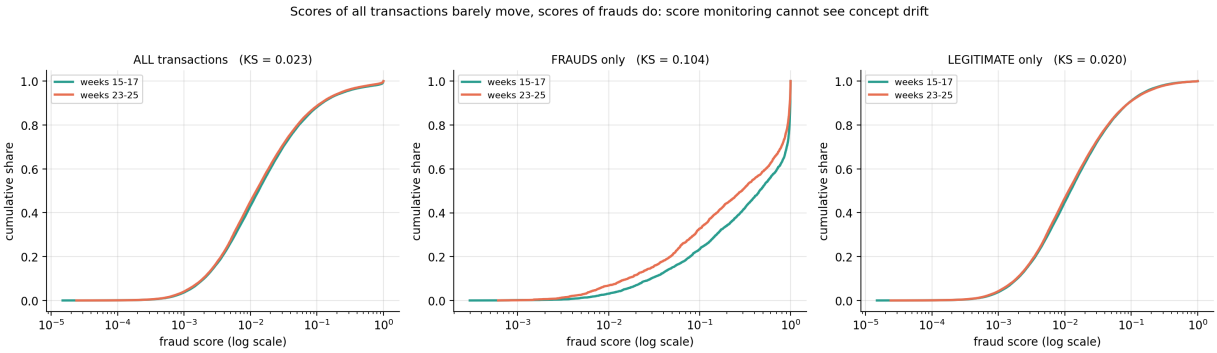


Figure 5. Cumulative score distributions of the first LightGBM in weeks 15–17 versus 23–25, for all transactions, frauds only and legitimate only.

The features that drift most are also not the ones the model relies on (Figure 6): $C7$ and $C13$ carry about 9.5% and 8.9% of the model's gain with no excess drift at all, whereas the strongest drifters (M columns and $D11$) each carry less than 0.5%. This is why a feature-drift alarm on the important features stayed silent.

Observations

- The main input drift is a change in *availability* (about 70% → 30% missing), not a change in observed values.
- The fraud rate inside the shrinking "$V4$ missing" group more than doubled.
- Scores of all transactions are stable (KS 0.023) but scores of frauds shift (KS 0.104): concept drift that score monitoring cannot see.
- The strongest drifters are not important to the model, and the important features show no excess drift.

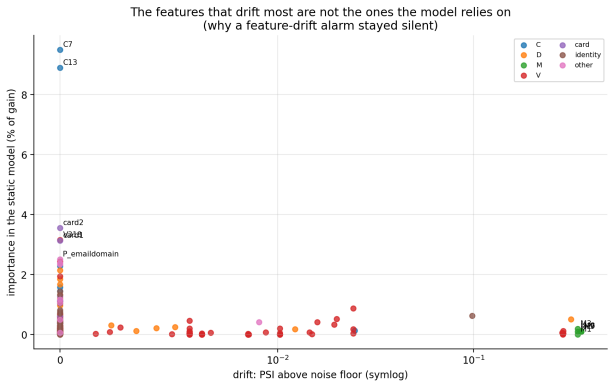


Figure 6. Drift (PSI above noise floor, symlog axis) against importance in the static model, by feature family.

6.3 Retraining and label delay (RQ4)

Table 6 and Figure 7 compare strategies. Retraining recovers about two thirds of the late-week loss: PR-AUC in weeks 23–25 is 0.55–0.56 versus 0.466 for the static model, with the same level in the early weeks. A paired bootstrap on future transactions gives gains over static of +0.040 (every 4 weeks), +0.051 (every 2 weeks) and +0.047 (rolling), all with intervals excluding zero and winning in 7, 9 and 9 of 11 weeks. For weeks 23–25 the gains are +0.084, +0.092 and +0.088 (computed from unrounded scores, so they can differ from rounded table values by 0.001–0.002).

Table 6. Retraining strategies on future weeks 15–25 (LightGBM, lr 0.1). Gain: PR-AUC difference to static with bootstrap 95% interval.

Strategy	trainings	PR-AUC	wk 23–25	Recall@5% (late)	gain vs static
static	1	0.505	0.466	0.519	–
every 4 weeks	3	0.545	0.550	0.594	+0.040 [0.036, 0.044]
every 2 weeks	6	0.556	0.558	0.605	+0.051 [0.047, 0.055]
rolling 8 wk, every 2	6	0.552	0.556	0.600	+0.047 [0.040, 0.055]
trigger > 10% drop	3	0.547	0.550	0.594	–
trigger > 15% drop	3	0.543	0.550	0.591	–
trigger > 25% drop	2	0.535	0.522	0.564	–

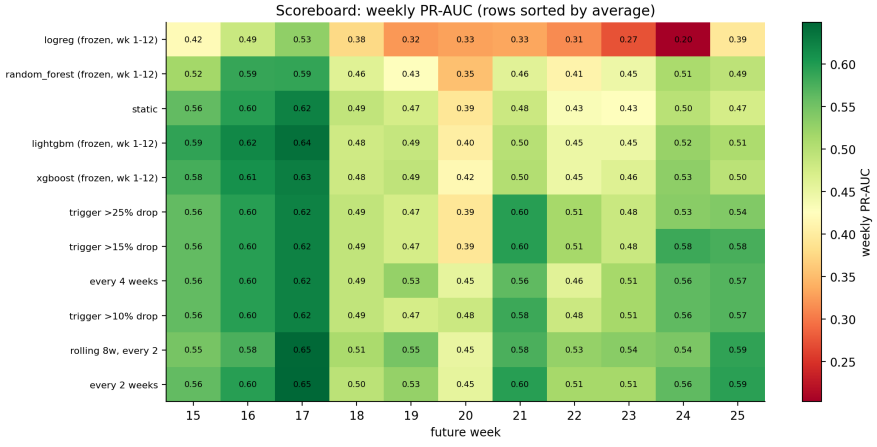


Figure 7. Scoreboard: weekly PR-AUC of every frozen model and retraining strategy (rows sorted by average). "lightgbm (frozen)" is the first model (lr 0.05); "static" is the faster version used in all retraining experiments (lr 0.1).

Doubling the retraining frequency from 4 to 2 weeks adds only 0.010 (95% interval 0.007–0.013), about two seed-noise units, so it is "slightly better, not clearly better": most of the benefit comes from the first few retrains (Figure 8, left). The rolling 8-week window uses about 170 000 rows per model (against 290 000–500 000) yet matches the expanding window, suggesting that *recency* matters more than data volume.

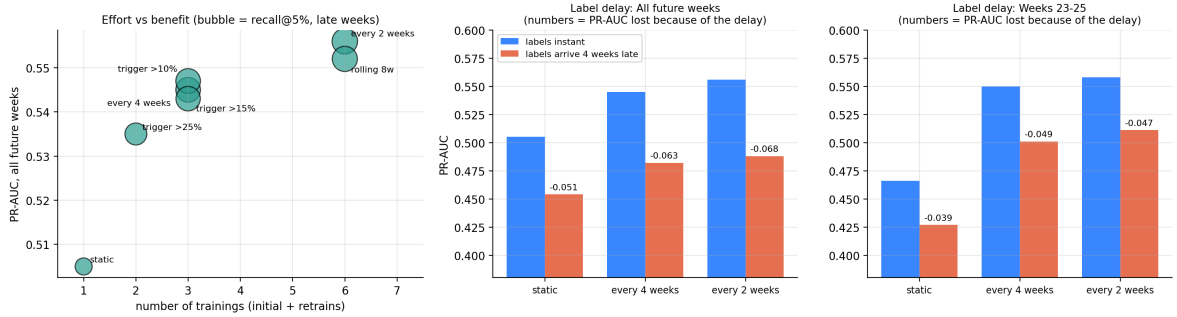


Figure 8. Left: PR-AUC against the number of trainings (bubble size = Recall@5%, late weeks). Middle and right: loss of PR-AUC when labels arrive 4 weeks late.

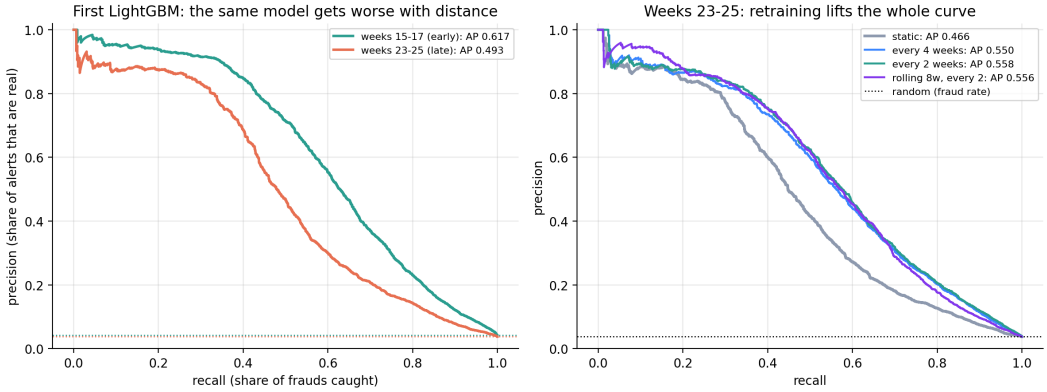


Figure 9. Precision–recall curves. Left: the first LightGBM, early versus late weeks. Right: weeks 23–25, static versus retrained.

At a fixed threshold chosen on validation, late-week precision/recall/F1 move from 0.592/0.407/0.482 (static) to 0.677/0.450/0.540 (every 4 weeks) and 0.654/0.478/0.552 (every 2 weeks) at about the same alert rate (25–28 per 1000), so retraining improves both precision and recall (Figure 9). For a review budget of 5% of transactions, the static model catches about 52% of late-week frauds against about 60% for the retrained models.

Triggers. Drift-triggered retraining matched a calendar with the same number of retrains and was not better; the cautious 25% trigger fired once and lost most of the benefit. A trigger can only react after the loss has occurred, and label-free triggers are excluded by Proposition 1.

Label delay. If a label becomes usable only 4 weeks after the transaction, every model trains on older data. The static model drops from 0.505 to 0.454 (part of this is simply less training data), and the gains of retraining shrink from $+0.040/+0.051$ to $+0.027/+0.034$ (every 4/2 weeks) overall, but remain large in weeks 23–25 (PR-AUC 0.427 static versus 0.501 and 0.511). The no-delay results are therefore optimistic by roughly a third, qualitatively as in Dal Pozzolo et al. [4].

Observations

- Retraining recovers about two thirds of the late-week loss; every retrained strategy beats static in the late weeks.
- Going from every 4 to every 2 weeks gives only +0.010: most gain comes from the first retrains.
- Week 20 is the weakest week for every model (static 0.39), and retrained models recover from week 21 onward.
- Label delay costs 0.05–0.07 PR-AUC overall but retraining still pays off in the late weeks.

6.4 Financial cost (RQ5)

Costs follow the cost equation of Section 4.5 with the scenarios of Table 2 and validation-chosen thresholds. Even the static model is worth deploying: it is 35–78% cheaper than flagging nothing. Table 7 shows that retraining every 4 or 2 weeks lowers cost in every scenario (7–17%), so the conclusion does not hinge on one assumption.

Table 7. Estimated cost saving relative to the static model (%). Last row: static vs. no model.

Strategy	cheap review	balanced	friction-heavy	fraud-heavy
every 4 weeks	7.4	12.2	10.5	8.9
every 2 weeks	11.2	17.3	13.9	12.6
rolling 8 wk, every 2	2.9	13.6	11.7	4.9
trigger $> 10\%$	6.0	9.8	9.0	6.9
trigger $> 15\%$	6.0	10.7	8.8	7.1
static vs. no model	78.4	52.9	34.7	76.4

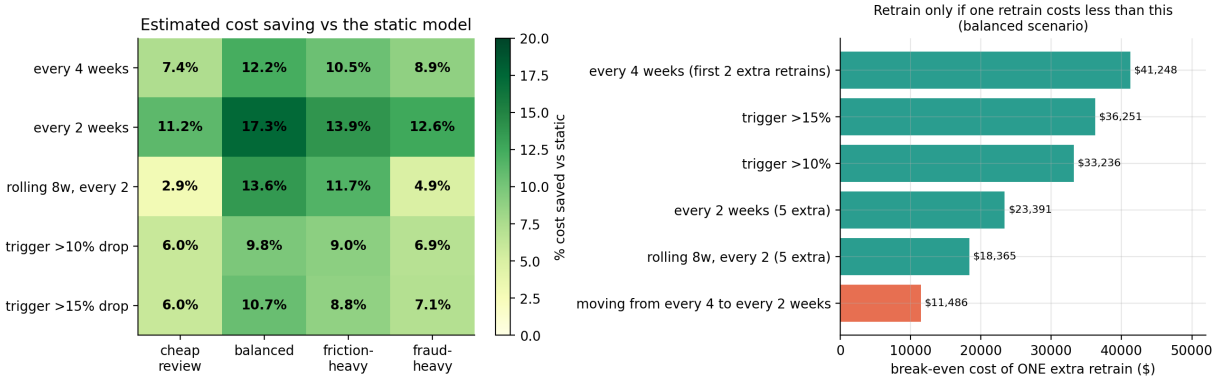


Figure 10. Left: estimated cost saving against the static model. Right: break-even cost of one extra retrain (balanced scenario).

In the balanced scenario the static model costs \$676 735 over the 11 weeks, and retraining every 2 weeks saves about \$117 000 (Figure 11). The break-even cost per extra retrain is \$41 248 for the first two extra retrains (every 4 weeks) but only about \$11 500 for the three further ones that move from every 4 to every 2 weeks, $(594\,240 - 559\,782)/3$. Retraining every 2 weeks therefore pays off only if one retrain costs less than about \$11 500.

The rolling window saves much less than the expanding window in some scenarios (2.9% vs. 11.2% when reviews are cheap) although its PR-AUC is similar. We have no tested explanation, which shows that PR-AUC alone is insufficient to rank strategies. There are no intervals on costs, so differences of a few percent between strategies should not be over-read.

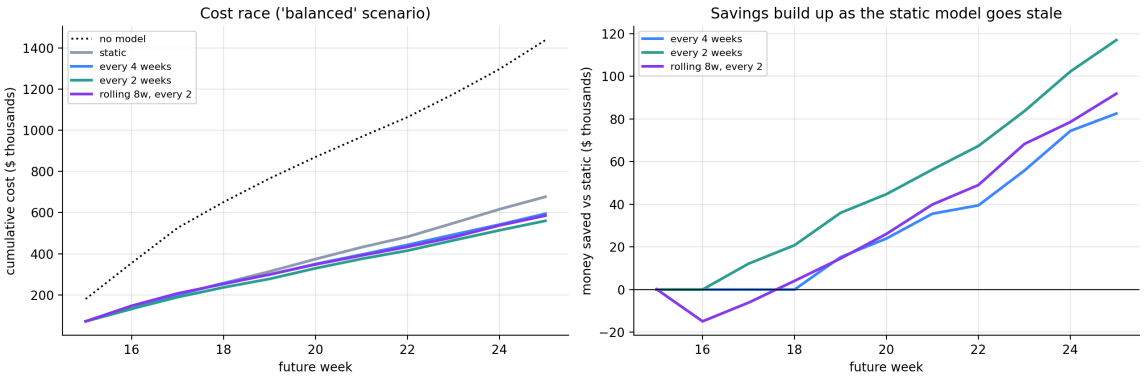


Figure 11. Cumulative cost (balanced scenario) and money saved against the static model over the future weeks.

Observations

- The static model is 35–78% cheaper than no model, depending on the scenario.
- Retraining every 4 or 2 weeks saves a further 7–17%; savings build up steadily as the static model goes stale.
- The first retrains are worth about $41 000 each; moving to every 2 weeks is worth only about $11 500 per extra retrain.

6.5 Explanation stability (RQ6)

SHAP importance of the static model is spread thinly: the top feature `TransactionAmt` holds only 4.8% of total importance, then *C*13 (4.7%), `card1`, `card2` (2.9% each) and *V*90 (2.6%). Across periods the ranking is almost unchanged (Table 8, Figure 12): Spearman 0.992 (early) and 0.986 (late) versus a noise baseline of 0.999, and 19 of the top-20 features are shared. Importance by feature family moves by about one point (*V*: 25.2% → 26.5%).

Table 8. Stability of mean |SHAP| importance of the static model vs. the training period (Spearman over 207 features, top-20 Jaccard).

Comparison	all rows	fraud rows
train vs. train (noise baseline)	(0.999, 1.000)	–
train vs. early test (wk 15–17)	(0.992, 0.905)	(0.989, 0.818)
train vs. late test (wk 23–25)	(0.986, 0.905)	(0.988, 0.905)
static vs. retrained (late rows)	(0.930, 0.905)	–
static vs. other seed (late rows)	(0.932, 0.818)	–

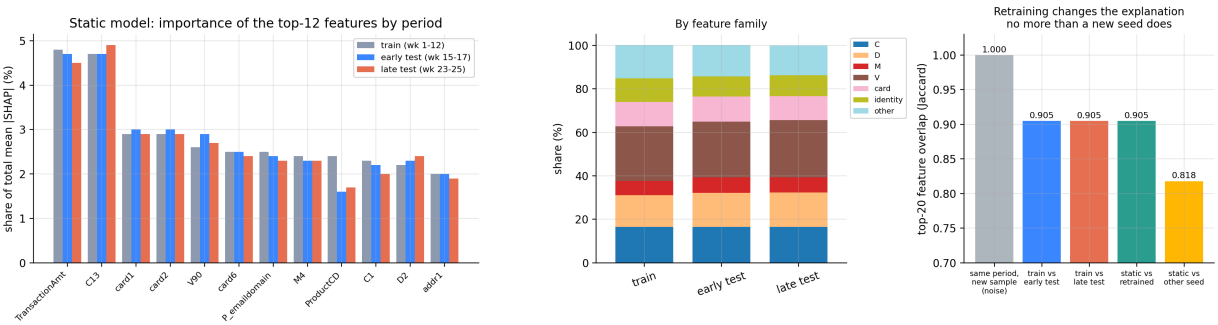


Figure 12. Left: importance of the top-12 features by period. Middle: importance by feature family. Right: top-20 overlap between periods and models, with a different seed as the noise reference.

The direction of effect is also stable (Spearman between feature value and SHAP value, training/early/late): `TransactionAmt` $+0.91/+0.90/+0.89$; $C13$ $-0.92/-0.92/-0.93$; $V90$ $-0.82/-0.83/-0.83$. The model retrained at week 25 differs from the static one by Spearman 0.930 and top-20 overlap 0.905, *no more* than a copy trained with another seed (0.932, 0.818). It would therefore be wrong to claim that retraining makes the model use different features. This is expected from theory: for a frozen f, the importance depends only on f and $P_t(x)$, so like the score distribution it is invariant under pure concept drift (the argument of Proposition 1).

Observations

- Explanations of the static model hardly change between training and the late future weeks.
- Retraining changes the explanation no more than a different random seed does.
- The fine structure that retraining improves is not visible in global importance.

6.6 The regression family and why trees win (RQ7)

All numbers in Table 9 are PR-AUC on the future weeks unless stated; settings marked "val" were chosen on the validation weeks (two weeks, 1 602 frauds, so differences of about 0.01 or less are not separable).

Table 9. Regression-type and tree models on the future weeks. Rel. drop $=$ (early $-$ late)/early.

Model	ROC	PR-AUC	wk 15–17	wk 23–25	rel. drop	note
LR, no missing flags	0.860	0.348	0.482	0.261	46%	not conv.
LR + flags, L2 (C=0.1)	0.862	0.389	0.483	0.361	25%	not conv.
LR + flags, L2 (val, C=0.001)	0.862	0.394	0.482	0.375	22%	not conv.
LR + flags, L1 (val, C=0.003)	0.863	0.391	0.479	0.378	21%	170/376 coef.
LR + flags, elastic net (saga)	0.839	0.379	0.454	0.336	17%	not conv.
Linear probability (ridge)	0.826	0.384	0.478	0.363	24%	
WoE binning + L2	0.853	0.365	0.464	0.377	28%	192 cols
Naive Bayes	0.763	0.089	0.106	0.087	18%	
Single tree, depth 6	0.824	0.303	0.364	0.307	16%	
Boosting, depth 1	0.863	0.389	0.493	0.352	29%	additive
Boosting, depth 2	0.881	0.471	0.559	0.454	19%	
Boosting, depth 3	0.888	0.498	0.580	0.489	16%	
Boosting, depth 6	0.888	0.508	0.593	0.475	20%	
Hybrid (300 trees, depth 5, val)	0.865	0.489	0.571	0.463	19%	C=0.003
Same trees alone	0.889	0.505	0.596	0.479	20%	
LightGBM (full)	0.894	0.523	0.617	0.493	20%	
Random Forest	0.885	0.487	0.564	0.483	15%	

Missing indicators explain most of logistic regression's weakness. Adding indicator columns lifts PR-AUC from 0.348 to about 0.39 and the late-week score from 0.261 to 0.36–0.38, while the early weeks are unchanged (0.482 vs. 0.483). The improvement sits where the missing pattern shifted (Section 6.2); a direct causal test was not done. Several logistic fits hit the iteration limit ("not conv."), so those scores carry some uncertainty; a likely cause is duplicated indicator columns (untested).

Penalties do not help, as the bias–variance split suggests. L2 and L1 with validation-chosen strength score 0.394 and 0.391 (a gap below seed noise), and the earlier fixed C values give 0.389–0.392; L1 keeps 170 of 376 columns at no loss, so many columns are redundant. The elastic-net run did not converge and is inconclusive. The limit is the approximation term, not the estimation

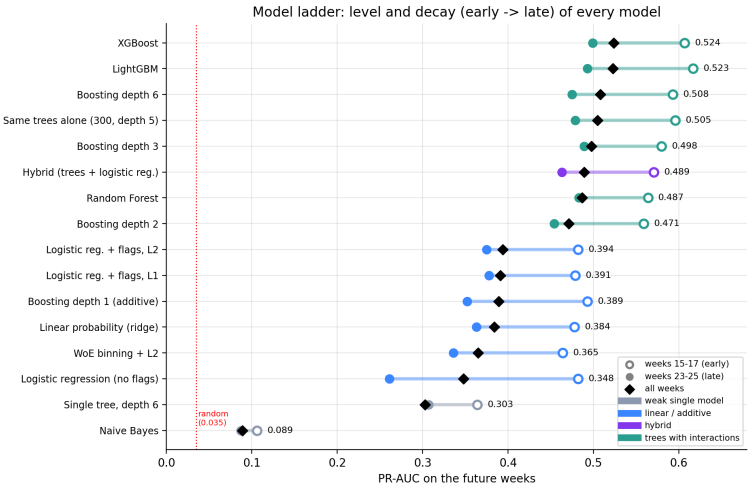


Figure 13. Model ladder: PR-AUC on all future weeks (diamond), early weeks 15–17 (open circle) and late weeks 23–25 (filled circle).

term. WoE binning is *worse* (0.365, with larger decay 28%); plausible reasons are the in-sample target encoding and bin scores frozen under a shifting missing pattern, but neither was tested. Information values are large (101 of 207 features have IV > 0.1; top: $V90$ 0.553, $V69$ 0.544, $V29$ 0.537) but are computed in-sample and include the missing bin, so they reflect the regime changes rather than pure predictive strength.

Interactions explain the gap (Proposition 2 and the depth argument). Depth-1 boosting, an additive model that handles missing values natively, equals logistic regression with indicators (0.389 vs. 0.389). Depths 2 and 3 close about 70% and 92% of the gap to depth 6 (0.471, 0.498, 0.508), and going from depth 3 to 6 adds less than two noise units (Figure 14, left). This matches the predicted interaction order. Single weak models fail for identifiable reasons: Naive Bayes (0.089) double-counts correlated evidence, and one depth-6 tree (0.303) is a high-variance single partition that boosting stabilises (settings differ, so this comparison is not clean).

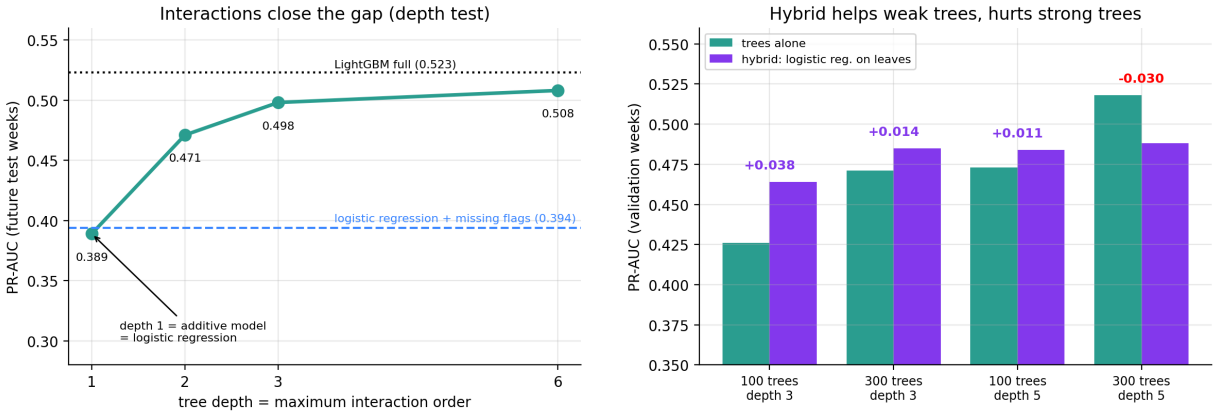


Figure 14. Left: PR-AUC against tree depth (maximum interaction order). Right: the hybrid versus its own trees on the validation weeks.

Hybrid. With 100 trees of depth 3 the hybrid scored 0.464 against 0.450 for its own trees on the test weeks (hybrid with $C = 0.01$, single run). Tuned on validation (Table 10), the largest tree set was selected and reached 0.489 on the test weeks, about three quarters of the way from penalised logistic regression (0.39) to LightGBM (0.523), but below its own trees (0.505). The regression adds value only when the trees are small (validation gains $+0.038$, $+0.014$, $+0.011$) and hurts at the largest size (-0.030), in line with the idea that boosting already optimises the leaf weights. We replicate the direction of He et al. [10] for weak tree sets, not for strong ones, on a different

problem. The strongest penalty ($C = 0.003$, the grid edge) was best at every size, so the penalty matters once thousands of correlated rule columns are used.

Table 10. Hybrid tuning on the validation weeks (AP), best $C = 0.003$ per size.

Trees, depth	100, 3	300, 3	100, 5	300, 5
leaf columns	787	2 336	2 817	8 104
trees alone (val)	0.426	0.471	0.473	0.518
hybrid (val)	0.464	0.485	0.484	0.488

Finally, every model that handles missingness decays by about 20% (15–29%); only the original logistic regression (46%) is an outlier, suggesting that the decay is a property of the data and not of the model family.

Observations

- Missing-value indicators fix most of logistic regression's problem (0.348 → 0.39), mainly in the late weeks.
- L1, L2 and elastic-net penalties give no gain: the limit is missing interactions, not variance.
- Boosting depth 1 equals logistic regression (0.389); depth 2–3 closes most of the gap to trees.
- The hybrid helps small tree sets (up to +0.038) and hurts the largest (−0.030).

6.7 Regression models of the drift itself (RQ2, RQ7)

Fraud rate. A Poisson trend gives a weekly rate ratio of 1.0084 ($p < 0.001$), but the data are heavily overdispersed ($\hat{\phi} = 34.8$): quasi-Poisson (95% interval 0.997–1.020, $p = 0.14$) and negative binomial (1.0051, 0.994–1.016, $p = 0.36$) show no reliable trend. The picture is a *level shift*, not a trend (Figure 15): with indicators at weeks 4 and 13 (Table 11) the trend vanishes (0.997, $p = 0.77$) and the rate steps up by a factor of 1.75 at week 4 (1.43–2.14) with no change at week 13 (0.92, 0.75–1.14).

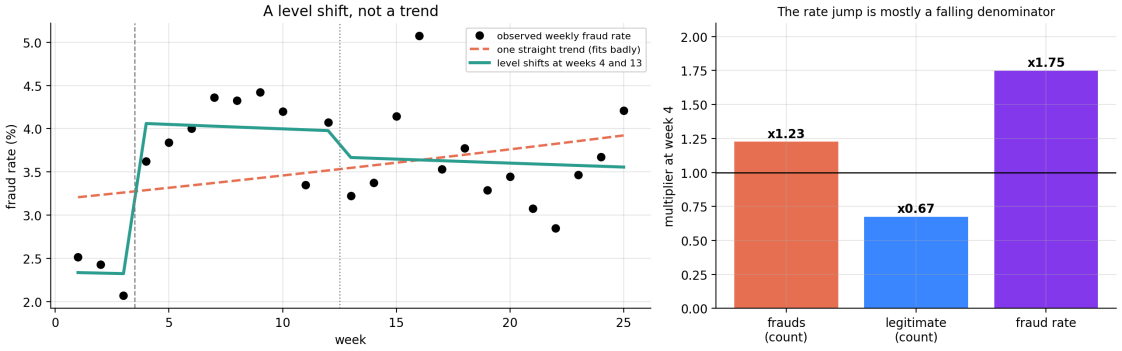


Figure 15. Left: weekly fraud rate with a single straight trend versus level shifts at weeks 4 and 13. Right: multipliers at week 4 for fraud count, legitimate count and fraud rate.

The step is mostly a falling denominator: legitimate weekly volume drops to 0.675 of its former level (0.59–0.77) while frauds rise modestly to 1.226 (1.005–1.496, $p = 0.044$). Indeed $\ln 1.226 - \ln 0.675 = 0.597$ ($e^{0.597} = 1.82$, close to the fitted 1.75), of which about 66% comes from the denominator. Why volume fell is not explained by the data. Inside the $V4$ groups the missing group steps by 2.14 (1.69–2.70) while the present group is flat (1.21, $p = 0.17$); a trend fitted to a step falsely suggests a rise of 4.1% per week in the missing group, so we do not report it as a trend.

Table 11. Count/rate models with change points (multipliers, 95% intervals; quasi-Poisson).

Outcome	step at week 4	step at week 13	weekly trend
Frauds (counts)	1.226 (1.005–1.496)	–	0.986 (0.977–0.995)
Legitimate (counts)	0.675 (0.591–0.771)	–	0.994 (0.988–1.001)
Rate, all	1.750 (1.430–2.141)	0.924 (0.747–1.143)	0.997 (0.981–1.014)
Rate, $V4$ missing	2.136 (1.687–2.703)	1.232 (0.940–1.615)	1.005 (0.984–1.027)
Rate, $V4$ present	1.206 (0.922–1.578)	0.999 (0.791–1.262)	0.998 (0.981–1.017)

Coefficient drift. We refit one logistic regression (59 design columns built from 35 of the 40 features the static model uses most, the other five being high-cardinality text columns; fixed scaling) on six 4-week windows. The fitted relationship moves steadily (Table 12, Figure 16): the share of coefficients that changed by more than $3\hat{\sigma}\sqrt{2}$ relative to weeks 1–4 grows from 31% to 47%, and the rank similarity of coefficient vectors falls with distance (neighbouring windows 0.67–0.80; first vs. last 0.51; the last two windows 0.78). Six clearly non-zero coefficients change sign (e.g. `card2` missing: $+1.46$ in weeks 9–12 to -0.80). Contrary to our prediction, missing-indicator coefficients change no more than value coefficients (9 of 20 vs. 19 of 39).

Table 12. Coefficient drift across 4-week windows.

Window	changed vs. weeks 1–4	changed vs. previous	Spearman with weeks 1–4
weeks 5–8	31%	31%	0.80
weeks 9–12	39%	31%	0.75
weeks 13–16	44%	39%	0.74
weeks 17–20	39%	31%	0.67
weeks 21–24	47%	31%	0.51

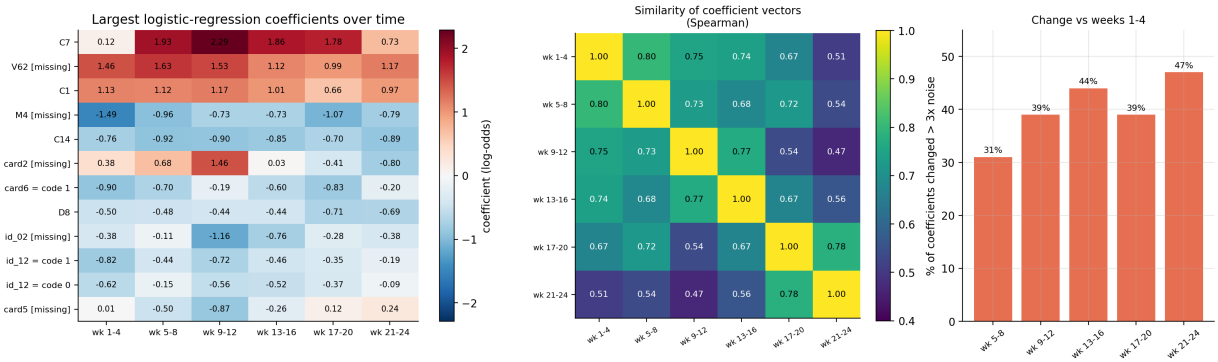


Figure 16. Left: largest logistic-regression coefficients per window. Middle: Spearman similarity of coefficient vectors. Right: share of coefficients changed by more than three times the noise, relative to weeks 1–4.

This is direct, regression-form evidence of concept drift and explains why retraining helps while the SHAP importance of a frozen model does not move. Cautions: coefficients of correlated features are entangled (the $V62$ value flips while $V62$-missing stays near $+1.1$ to $+1.6$), $\hat{\sigma}$ is optimistic (repeated cards), $3\hat{\sigma}\sqrt{2}$ flags small changes at 10^5 rows per window, and the first window straddles the week-4 step. Read the overall pattern, not single coefficients.

Observations

- The fraud rate shows a *level shift* at week 4, not a trend; a straight line fits badly.
- About 66% of the rate jump comes from fewer legitimate transactions and 34% from more frauds.
- The relationship between features and fraud changes steadily: 31% $\rightarrow$ 47% of coefficients differ from weeks 1–4, and several change sign.

7 Discussion and observation table

Table 13 links each result to a reason and states how strongly the reason is supported: *theory* (follows from Section 4), *tested* (an experiment isolates it), *partly* (consistent but not isolated) or *untested*.

Table 13. Results, proposed reasons and evidence level.

Observation	Reason	Evidence
Random split 56% too optimistic	Random mixing puts the same days and cards in train and test; the model sees the "future"	Effect tested; mechanism untested
Trees beat logistic regression	Interactions: an additive logit cannot express them (Prop. 2); depth-1 boosting = LR, depth 2–3 closes the gap	Theory + tested
LR improves with missing flags (late weeks)	Missingness is informative and its pattern shifted	Partly (no direct test)
Penalties do not help	They reduce estimation error only; the limit is approximation	Theory + partly tested
WoE worse than flags	In-sample bin scores, frozen under a shifted missing pattern	Untested
Naive Bayes worst; single tree weak	Double-counted correlated evidence; high-variance single partition	Theory; untested on data
Hybrid helps only with small trees	Boosting already optimises leaf weights	Partly (one run per size)
Label-free monitors silent	Score and feature laws depend on $P_t(x)$ only (Prop. 1); $P_t(y \mid x)$ changed (coefficient drift); frauds' scores shift (KS 0.104) while all scores do not (0.023)	Theory + tested
Strongest drifters are unimportant	Drifting columns (M, $D11$) carry $< 0.5\%$ of gain; important columns ($C7$, $C13$) show no excess drift	Tested
Frozen-model SHAP stable, accuracy falls	Importance depends on f and $P_t(x)$ only	Theory + tested
Retraining recovers about 2/3 of the loss; rolling $\approx$ expanding	Recent labels carry the current relation; recency beats volume	Tested (not isolated)
Triggers $\approx$ calendar	A label-based trigger reacts only after the loss; same number of retrains	Partly
Rolling window worse on cost	–	Unexplained
Rate jump at week 4	Mostly fewer legitimate transactions (66%), modestly more fraud (34%)	Tested (decomposition); cause unknown
Missing share falls (weeks 4, 13)	Change in data collection or merging (hypothesis)	Untested

Expectations that failed. Reporting these matters as much as the confirmations.

(i) Label-free drift signals did not detect the accuracy loss.
(ii) The rolling window was not better than the expanding window.
(iii) The fraud rate did not change at week 13.
(iv) Penalties did not help logistic regression.
(v) Retraining did not change SHAP importance beyond seed noise.
(vi) Missing-indicator coefficients were not special in the coefficient-drift analysis.
(vii) The hybrid did not beat strong trees.

A consistent picture. The data change in how complete they are (availability) and in volume, and the relationship between features and fraud also shifts gradually (coefficient drift). A frozen model keeps the same global behaviour (stable scores and explanations, Proposition 1) but is increasingly wrong where the relation has moved, so accuracy falls by about 20%, while models refitted on recent labels track the relation. Because only labels reveal the problem, monitoring must be label-based, with the delay accounted for.

8 Limitations

1. One 26-week history; weekly blocks are correlated and not independent samples.
2. Bootstrap intervals treat transactions as independent although cards repeat, so true uncertainty is larger; the coefficient-drift noise is likewise optimistic.
3. Costs are assumed, and no intervals are given for them.
4. Most features are masked, so drift can be described but not explained in business terms; the cause of the week-4 volume drop and of the availability steps is unknown.
5. Several choices were made after seeing results: the $V4$ split, the change points at weeks 4 and 13, and the trigger thresholds (all reported). Hyper-parameters of the early logistic-regression variants were compared on test numbers; the final penalty and hybrid choices use validation, which is only two weeks long.
6. Seed variation was measured only for the static LightGBM, and several logistic fits did not converge.
7. Labels were instantaneous in the main experiments; only one delay (4 weeks) was tested.
8. The "future" is the last part of the labelled period, so no hidden-test comparison is possible. We did not implement error-based drift detectors [8] or the two-classifier scheme of Dal Pozzolo et al. [5]; both are natural extensions.

9 Conclusion

Evaluated chronologically, a fraud model that looks excellent under a random split (PR-AUC 0.816) scores 0.523 on the future and loses about a fifth of that within weeks. The drift is visible as a change in feature availability and in the fitted regression coefficients, but score- and importance-based monitors cannot see it, as a push-forward argument and the score distributions of frauds show.

Retraining on recent labelled data recovers about two thirds of the loss and lowers estimated cost by 7–17% when done every 2 or 4 weeks in every assumed scenario, with most of the benefit from the first few retrains and a smaller benefit under label delay. For regression models, penalties do not remove the main limitation of logistic regression on this data, the lack of interactions; the tree–regression hybrid recovers most of the gap but does not beat its own strong trees.

In practice the results argue for label-based monitoring, a retraining schedule chosen by the cost of one retrain, and explicit tracking of feature availability.

References

[1] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[2] A. C. Cameron and P. K. Trivedi. *Regression Analysis of Count Data*. Cambridge University Press, 2nd ed., 2013.

[3] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD*, 2016.

[4] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi and G. Bontempi. Credit card fraud detection and concept-drift adaptation with delayed supervised information. In *IJCNN*, 2015.

[5] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi and G. Bontempi. Credit card fraud detection: A realistic modeling and a novel learning strategy. *IEEE Trans. Neural Networks and Learning Systems*, 2018.

[6] C. Elkan. The foundations of cost-sensitive learning. In *IJCAI*, 2001.

[7] J. H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.

[8] J. Gama, P. Medas, G. Castillo and P. Rodrigues. Learning with drift detection. In *SBIA 2004*, 2004.

[9] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy and A. Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), 2014.

[10] X. He et al. Practical lessons from predicting clicks on ads at Facebook. In *ADKDD'14*, ACM, 2014.

[11] IEEE Computational Intelligence Society and Vesta Corporation. IEEE-CIS Fraud Detection (Kaggle competition), 2019. `https://www.kaggle.com/competitions/ieee-fraud-detection`

[12] G. Ke et al. LightGBM: A highly efficient gradient boosting decision tree. In *NeurIPS 30*, 2017.

[13] S. Kullback and R. A. Leibler. On information and sufficiency. *Annals of Mathematical Statistics*, 22(1):79–86, 1951.

[14] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *NeurIPS 30*, 2017.

[15] T. Saito and M. Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3):e0118432, 2015.

[16] N. Siddiqi. *Credit Risk Scorecards*. Wiley, 2006.

[17] R. Tibshirani. Regression shrinkage and selection via the lasso. *J. Royal Statistical Society B*, 58(1):267–288, 1996.

[18] H. Zou and T. Hastie. Regularization and variable selection via the elastic net. *J. Royal Statistical Society B*, 67(2):301–320, 2005.

Code, notebooks and data availability

All source code, notebooks, step files and saved models are available in the project GitHub repository:

GitHub: https://github.com/Kanishk-Bhuradiya/adaptive-fraud-detection/tree/main

Kaggle dataset-audit notebook: https://www.kaggle.com/code/kanishkbhuradiya/01-dataset-audit

IEEE-CIS Fraud Detection dataset: https://www.kaggle.com/competitions/ieee-fraud-detection/data

Supplementary bundle (initial code outputs and intermediate results):
https://drive.google.com/file/d/11cN9a-m2gUzDWRSY9hMPJ5nPtj_OVLnr/view?usp=sharing

Reproducibility and access note: All analyses were run in Kaggle notebooks; the competition data were accessed directly on Kaggle and were not downloaded locally. The Drive bundle contains selected early outputs and intermediate results, not the full competition dataset. Access to the full IEEE-CIS data requires a Kaggle account and registration for the competition, including acceptance of its rules.